CCPA Compliance System

OPEN HACK 2026

CSA, IISc

1. Problem Statement

Motivation

Data privacy regulation is one of the most consequential areas of modern technology law. The California Consumer Privacy Act (CCPA) establishes strict rules around how companies collect, use, share, and delete consumers' personal information. Yet many organizations — intentionally or not — engage in practices that directly violate these rules, exposing consumers to real harms such as unauthorized data sales, ignored deletion requests, and discriminatory pricing for users who exercise their privacy rights.

Manually reviewing business practices against dense legal text is time-consuming and error-prone. There is a clear need for intelligent, automated systems that can read a description of a company's data handling behavior, reason over the relevant law, and instantly flag potential violations along with the specific statute sections at issue. Building such a system requires combining natural language understanding, legal knowledge, and reliable structured output — a challenging and impactful engineering problem.

The Challenge

Your challenge is to build an intelligent system that, given a short natural-language prompt describing a business practice, automatically determines whether that practice violates any section of the CCPA statute and, if so, identifies the exact article(s) violated. **You are free to design and implement your solution however you see fit** — the approach, architecture, and tools are entirely your choice.

The system must be containerized with Docker and expose a FastAPI HTTP server. An automated test script (**test.py**) will evaluate your container by sending prompts and checking the structured JSON responses.

What Your System Must Do

- Accept a natural-language prompt describing a company's data practice.
- Analyze the prompt against the CCPA statute provided to you.
- Classify whether the described practice violates the CCPA.
- Return a strictly-formatted JSON response — no extra text, no explanation.

2. Input / Output Format

POST /analyze — Request Body

```
{
    "prompt": "<natural language description of a business practice>"
}
```

POST /analyze — Response Body

```
{
    "harmful": true | false,
    "articles": ["Section 1798.xxx", ...]
}
```

Response Rules

Field	Type	Rule
harmful	boolean	true if the prompt describes a CCPA violation, false otherwise
articles	list of strings	Non-empty when harmful=true. Must be [] when harmful=false.

Example — Violation Detected

```
// Request
{"prompt": "We sell customer browsing history to ad networks without notifying them."}

// Response
{"harmful": true, "articles": ["Section 1798.100", "Section 1798.120"]}
```

Example — No Violation

```
// Request
{"prompt": "We provide a clear privacy policy and honor all deletion requests."}

// Response
{"harmful": false, "articles": []}
```

⚠ The response body must contain ONLY valid JSON. Any extra text, markdown, or explanation will cause the automated test to fail.

⚠ A prompt may contain more than one violation, so make sure to cite all relevant articles. Partial citations will receive partial credit, but citing any incorrect article will result in zero marks

3. Resources Provided

You will find the following files in the participant package:

File	Description
ccpa_statute.pdf	Full text of the California Consumer Privacy Act — use this as your primary knowledge source for the compliance system.
validate_format.py	Run it locally to verify your submission before the deadline. Note that this script validates the format of your output and it does not check the correctness of your citations.
participant_instructions.pdf	This document.

4. Technical Requirements

Language Model

- Maximum 8 billion parameters.
- If the model requires a Hugging Face token, supply it via the environment variable `HF_TOKEN` do NOT hardcode it.

Docker & API

- Your container must start with a single command.
- FastAPI server must listen on **port 8000**.
- Must expose both `GET /health` (returns HTTP 200) and `POST /analyze`.
- Must support: `docker run --gpus all -p 8000:8000 -e HF_TOKEN=<token> <image>`
- Do not use interactive prompts or require manual steps inside the container.

5. How validate_format.py Evaluates You

This script acts as a strict "syntax checker" to ensure your API behaves correctly. Here is exactly what the script does, step-by-step:

1. **Startup Check:** It pings your `GET /health` endpoint every 5 seconds for up to 5 minutes. This gives your container enough time to download and load the large LLM into memory.
2. **Sending Prompts:** It sends a mix of safe and harmful questions to your `POST /analyze` endpoint.
3. **JSON Validation:** It strictly checks that your API responds with valid JSON containing exactly two keys: `harmful` (a true/false boolean) and `articles` (a list).
4. **Logic Validation:** It enforces the basic formatting rules:
   - If `harmful` is `true`, your `articles` list must contain at least one item.
   - If `harmful` is `false`, your `articles` list must be completely empty (`[]`).
5. **Logging Results:** It saves your API's exact responses into a file so the organizers can review them.

> ⚠ validate_format.py does not check if your AI is actually correct. It does not compare your citations against the ground truth.As long as your system outputs correctly formatted JSON strings—even if your AI hallucinates the completely wrong CCPA article—this script will mark it as a PASS. Final grading will be done by the organizers using a separate, hidden script that strictly evaluates the accuracy of your retrieved legal articles.

6. Submission Requirements

Submit all of the following:

- **Docker Hub Image Link (MANDATORY):** Push your image to Docker Hub and share the public link. This is the primary evaluation artifact. Example: `docker push yourusername/ccpa-compliance:latest`

- **Source Code - ZIP Archive (MANDATORY):** Submit a ZIP archive of your complete project source code. Do NOT submit a GitHub repository link; the ZIP is the only accepted format.

- **README.md (MANDATORY):** Your README must be detailed and must cover all of the following sections:
  - **Solution Overview:** Briefly describe your approach and system architecture — what technique(s) you used to analyze CCPA compliance, which model(s) or libraries are involved, and how components fit together end-to-end (input → processing pipeline → JSON output).
  - **Docker Run Command:** The exact command to pull and run your container including all flags. Example: `docker run --gpus all -p 8000:8000 -e HF_TOKEN=xxx yourusername/ccpa-compliance:latest`
  - **Environment Variables:** List every environment variable your container requires with a description of each (e.g., HF_TOKEN — Hugging Face access token for gated models).
  - **GPU Requirements:** Minimum GPU VRAM required, and whether CPU-only fallback is supported.
  - **Local Setup Instructions (Fallback):** Step-by-step instructions to run your system directly on a Linux VM without Docker. This is used only if your Docker image fails to start — note that manual deployment will incur a score penalty. Include: Python version, pip install commands, how to start the FastAPI server, and how to call the API manually.
  - **API Usage Examples:** Show how to call POST /analyze and GET /health manually, with example curl commands and expected JSON responses.

- **Hugging Face Token (if required):** Provide a temporary read-only HF token if your chosen model is gated. Submit via the organizer's secure form — never include tokens in source code or in the ZIP archive.

⚠ If your Docker container fails to start but your code works when run manually, your deployment score will be penalized. Always test the full Docker flow before submitting.

7. Scoring Criteria

Criterion	Description
Correctness	Number of test cases where harmful/safe classification is correct (primary metric).
Article Citation Accuracy	Relevant CCPA sections correctly identified and returned for harmful prompts.
Solution Approach	Quality and creativity of your technical approach — how well your system reasons over the CCPA statute to make accurate decisions.
README & Documentation	Clarity and completeness of the README: solution workflow, docker run instructions, environment variables, local setup steps, and API usage examples.
Docker Deployment Stability	Container starts cleanly, serves all requests without crashing, and handles edge cases gracefully.

8. Common Failure Points — Avoid These!

- Missing or broken `GET /health` endpoint — server will be declared unready.
- Server listening on the wrong port (must be 8000).
- Returning `harmful` as a string ("true"/"false") instead of a boolean (true/false).
- Including explanation text in the response body — only JSON is allowed.
- Heavy initialization work (e.g., index building, model loading) done per-request instead of at startup — will cause severe latency.
- Model weights downloaded at inference time — pre-download during image build.
- Hardcoded API keys or HF tokens in source code.
- Interactive prompts or manual steps required inside the container.

✅ Best practice: Build your Docker image locally, run it with docker run --gpus all -p 8000:8000, then execute python test.py and confirm all tests PASS before submitting your ZIP.

Good luck! Build something accurate, well-documented, and properly containerized.